

82.1. Chapitre 13 — Ce qui va ensemble : ce qui vient

L'entrée du chapitre 13 : ce qu'on a vu jusqu'ici, et ce que ses 8 étapes vont ajouter, une à la fois.

1. Ce qu'il faut comprendre

D'où l'on vient. Le chapitre 12, « Les fonctions : nommer un geste », vient de se terminer. Tout ce qu'il a montré reste valable : on s'en sert à partir d'ici sans le réexpliquer.

Ce que ce chapitre apporte : Ce qui va ensemble. Il compte 8 étapes :

- Une « struct » : ce qui va ensemble
- Le décor qui défile
- La caméra : c'est le monde qui bouge
- Trois variables qui parlent de la même chose
- Un tableau de « struct »
- Une méthode : ce qu'elle coûte, ce qui reste refusé
- Les trois façons de faire la même chose
- Une méthode qui en appelle une autre

Aucune fonction nouvelle ici : ce chapitre n'emploie que des fonctions déjà présentées. Tout l'effort porte sur la façon d'écrire le programme.

Comment avancer : une étape à la fois. Lis l'explication, lance le programme, regarde ce qu'on doit voir, puis fais l'essai proposé. Si quelque chose t'échappe, l'étape d'avant contient la pièce qui manque.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **458 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	— la mise en route de la console
main()	29 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	— attend de pouvoir écrire une case
EcrireTexte	10 o	← texte()	— écrit un texte, lettre par lettre

```

CopierTuiles      94 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
Tuiles            176 o ← CopierTuiles – 11 lettres sur 43
"CHAPITRE 13"     11 o ← texte() – un texte gravé
"C EST PARTI"     11 o ← texte() – un texte gravé
Lettres gravées : ACEHIPRST13
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
    texte() × 2      22 o sur place, dans main() + EcrireTexte (10 o)
    image() × 1      3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- Chapitre 13 : Ce qui va ensemble ----
// L'entrée du chapitre : rien de neuf dans ce programme. Il annonce la suite.

#include <texte> // écrit un texte à l'écran

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

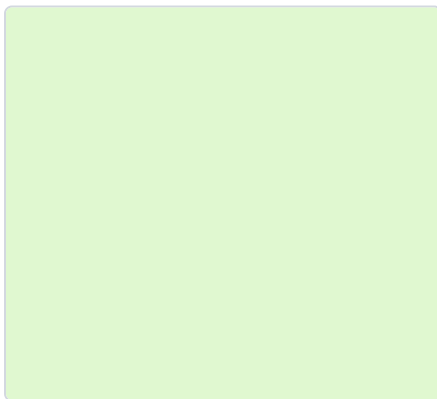
int main() {
    texte(1, 7, "CHAPITRE 13"); // le numéro du chapitre
    texte(1, 9, "C EST PARTI");

    while (true) {
        image();
    }
}

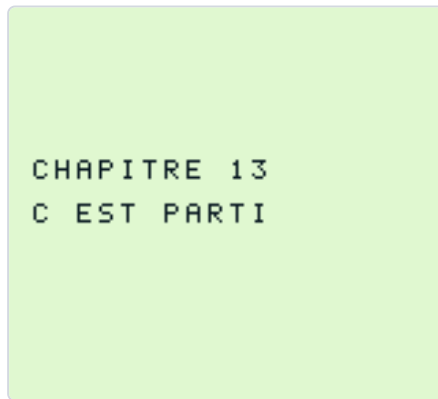
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

CHAPITRE 13, puis C EST PARTI.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le chapitre 13 s'annonce

Ensuite : **83. Une « struct » : ce qui va ensemble** — chapitre 13 : Ce qui va ensemble. · gameboy3 — écrire une cartouche Game Boy en C++